

ATLIQ TECHNOLOGIES

Enterprise AI Delivery — Forward Deployed Engineering

SOLUTION DESIGN DOCUMENT · HLD + LLD

AI-Powered Field Service Command Centre

Prepared for Vaayu Pumps & Systems Ltd

Document ID	VPS-FSCC-2026-SDD
Version	1.0
Status	Approved by client architecture and security
Date	23 February 2026
Author	Kabir Menon, Forward Deployed Engineer (AtliQ Technologies)
Reviewed by	Fahad Qureshi, Head of IT Infrastructure & Security; Ritu Balan, SAP PM Functional Lead (Vaayu, internal)
Derived from	VPS-FSCC-2026-BRD v1.1 (baselined 18 February 2026)
Classification	Client Confidential

ABOUT THIS DOCUMENT — READ BEFORE YOU USE IT

This is a TEACHING ARTEFACT from the Codebasics Forward Deployed AI Engineer Roadmap 2026. Vaayu Pumps & Systems Ltd, AtliQ Technologies and every person named in it are fictional.

It is the middle document of three. The BRD says what the business needs and why. This SDD says what the system IS — its components, the technology chosen and why, where it runs, and how it stays fast, cheap, safe and observable. The TDD then says exactly how each part BEHAVES.

Diagrams are drawn with Word tables and monospace text so the file stays self-contained. Annex A gives you the Mermaid and Eraser.io source for every one of them — paste it into a real tool and you get a proper diagram in seconds. That is how these are actually produced on a live engagement.

Document Control

Version history

Ver.	Date	Author	Summary of change	Reviewed by
0.2	11 Feb 2026	K. Menon	First architecture draft: component model and stack proposal.	—
0.6	16 Feb 2026	K. Menon	Deployment topology reworked after the security workshop. On-premise LLM hosting evaluated and rejected for Phase 1 — recorded as ADR-001.	F. Qureshi
0.8	19 Feb 2026	K. Menon	Added CR-01 impact from BRD v1.1: SAP MM stock interface, cache design and the parts shortfall path.	R. Balan
1.0	23 Feb 2026	K. Menon	Approved by client architecture and security. Sizing confirmed against BRD ASM-03. Issued as the baseline for the TDD.	F. Qureshi, R. Balan

NOTICE THE ORDER THESE THREE DOCUMENTS WERE WRITTEN IN

BRD baselined 18 Feb. SDD approved 23 Feb. TDD issued 27 Feb. Each waits for the one before it, and each is signed by a different person: the sponsor signs the BRD, client IT and security sign the SDD, the technical lead signs the TDD.

That sequence is not bureaucracy. It is how you avoid designing a system nobody agreed to, or building a design that security rejects in week eleven.

Related documents

Reference	Document	Status
01-C	AS-IS Process Map — Complaint to Closure (swim-lane)	Issued
VPS-FSCC-2026-BRD	Business Requirement Document	v1.1 — baselined 18 February 2026
VPS-FSCC-2026-TDD	Technical Design Document	v1.0 — issued 27 February 2026
VPS-FSCC-2026-SEC	Security Review Record and Penetration Test Report	Scheduled, Week 10
VPS-FSCC-2026-UAT	User Acceptance Test Script	Planned, Week 9

Contents

Document Control	2
Version history.....	2
Related documents.....	2
Contents.....	3
1. Purpose and Document Boundary	5
1.1 What lives where	5
1.2 The four documents, in one table.....	5
2. Solution Context.....	7
2.1 System context	7
2.2 Why this is not a classic architecture.....	7
2.3 Design principles	7
3. Logical Architecture	9
3.1 Layered component view	9
3.2 Component inventory.....	9
4. Technology Stack and Rationale	11
4.1 Stack decisions.....	11
4.2 Model routing policy.....	11
4.3 Rejected options	12
5. Deployment Topology.....	13
5.1 On-premise and cloud split	13
5.2 Network and egress.....	13
5.3 Environments	13
5.4 Sizing	14
6. Security and Data Protection Architecture	15
6.1 Threat model summary.....	15
6.2 Identity and access.....	15
6.3 Data protection.....	15
7. Availability, Degradation and Recovery	16
7.1 Availability targets.....	16
7.2 Degradation tiers	16
7.3 Why this architecture is deliberately single-site.....	16
8. Latency, Cost and Observability Architecture	18
8.1 Latency budget	18
8.2 Cost architecture.....	18

8.3 Observability and evaluation	19
9. Logical Data Model	20
9.1 Entity relationships	20
9.2 Entities owned by this system	20
9.3 Read models	20
10. Interface Inventory and Contracts	22
10.1 Internal API surface	22
10.2 External interface inventory	22
11. Sequence Designs	24
11.1 P1 complaint to work order — the approved path	24
11.2 SAP write failure — the degraded path	24
12. Agent and Tool Boundary Design	25
13. Configuration Inventory	26
14. Non-Functional Traceability	27
Annex A — Diagram Source	28
A.1 System context — Mermaid	28
A.2 Component architecture — Eraser.io DSL	28
A.3 P1 sequence — Mermaid	28
Annex B — Architecture Decision Records	30
ADR-001 — Model inference in Azure OpenAI Central India rather than on-premise	30
ADR-002 — A deterministic gate holds the only SAP write capability	30
ADR-003 — pgvector on the existing PostgreSQL cluster rather than a dedicated vector database	30
Annex C — SDD Reviewer's Rubric	32

1. Purpose and Document Boundary

This Solution Design Document defines the architecture of the **AI-Powered Field Service Command Centre**. It covers what the system is made of, why each technology was chosen, where each part runs, and how the cross-cutting concerns — security, data residency, availability, latency, cost and observability — are satisfied.

Part A (High-Level Design) is written for client architecture and security, who approve it. Part B (Low-Level Design) is written for the delivery team and is the structural input to the TDD.

Commented [SFR1]: This section exists because the question learners ask most is what goes in the SDD versus the TDD, and the honest answer is that it varies by organisation.
What does not vary: someone must own the shape of the system, and someone must own its exact behaviour. Agree the split in writing at kick-off. Arguing about it in week nine is how deliverables get written twice, or not at all.

1.1 What lives where

Concern	In this SDD	In the TDD
Components	What components exist, what each is responsible for, how they connect	What each one does with a given input
Data	Logical data model, entities and relationships, storage choice	Field-level validation rules, formats, null handling
Interfaces	Interface inventory, request and response models, protocol and direction	Error codes, retry policy, idempotency rules, failure behaviour
Business rules	That a rule engine exists, and where it sits	The actual rule tables — severity, SLA, ranking, parts
AI design	Which components are model-driven, the routing and caching strategy, tool boundaries, the deterministic gate	Model IDs, temperatures, prompt versions, guardrail specifics
Infrastructure	Deployment topology, network, environments, sizing, disaster recovery	Not covered
Decisions	Architecture Decision Records with rejected alternatives (Annex B)	Not covered

1.2 The four documents, in one table

Document	Answers	Signed by	Written when
BRD	What does the business need, and why is it worth doing?	Business sponsor	After discovery, before design
SDD — HLD	What is the system made of, and where does it run?	Client IT and security	After the BRD is baselined
SDD — LLD	What is the internal structure — data, interfaces, sequences?	Technical lead and client IT	With the HLD
TDD	Exactly how does each part behave, rule by rule?	Technical lead	After the SDD is approved

IF YOUR CLIENT USES DIFFERENT NAMES

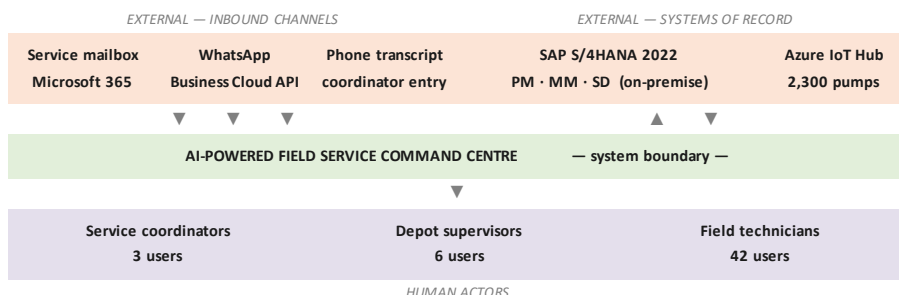
Some organisations write one SDD containing everything. Some split HLD and LLD into separate documents. Some call the LLD a TDD and have no separate SDD at all.

Do not spend political capital on the naming. Ask two questions instead: who signs off on the architecture, and who signs off on the behaviour? Then write whatever documents deliver those two signatures.

2. Solution Context

2.1 System context

The Command Centre sits between three inbound channels and the client's SAP estate. Everything outside the system boundary is owned by someone else — which is why the interface inventory in Section 10 names a client-side owner for every one of them.



2.2 Why this is not a classic architecture

A database query is deterministic, costs almost nothing, returns in milliseconds and never invents a row. A language model breaks all four assumptions. Each break forces a component into this architecture that a conventional system would not need.

LLM property	What it breaks	What this architecture does about it
Non-deterministic — the same input can produce a different output	You cannot assert equality in a test, and you cannot promise a client identical behaviour twice	Severity and ranking are deterministic code, not model output (Sections 3, 12). An eval harness with a golden set replaces equality assertions (Section 8.3).
Expensive per call — orders of magnitude more than a database read	Cost becomes a design constraint rather than an operations problem	A router sends cheap work to a small model and reserves the premium model for one task (Section 4.2). A three-tier cache absorbs repeat traffic (Section 8.2).
Slow — seconds, not milliseconds, and produced token by token	Synchronous request-response design stops working	Asynchronous pipeline with a queue; the coordinator is never blocked. Streaming on the console. Latency budget allocated per stage (Section 8.1).
Confidently wrong — it will invent an equipment number as readily as recall one	You cannot let it act unsupervised on a system of record	Every generated claim carries a citation or is discarded. A deterministic gate holds the only SAP write (DP-2, Section 12). Confidence below threshold routes to a human.

Commented [SFR2]: Classic system design assumes components are deterministic, cheap and fast. An LLM is none of those three, and it can be confidently wrong. Almost every unusual thing in this architecture — the router, the caches, the deterministic gate, the eval harness — exists to compensate for one of those four properties. If you can name which property each box is defending against, you understand the design.

2.3 Design principles

Eight principles govern every decision in this document. Where a later section makes a trade-off, it names the principle it is serving.

ID	Principle	Why, and what it traces to
DP-1	SAP remains the single system of record. This system never becomes a second source of truth.	A second source of truth in an ERP landscape is a reconciliation problem that outlives the project. BRD Section 2.1.
DP-2	No model-driven component holds an irreversible action. A deterministic gate sits in front of every write.	Makes prompt injection structurally unable to create a work order, rather than merely discouraged. BRD NFR-06, NFR-07.
DP-3	Fail towards a human, never towards silence. Every degraded path is announced.	A silent wrong answer in a service workflow costs more than a loud failure. BRD FR-009, NFR-10.
DP-4	Personal data stays in India. Only model inference egresses, and only to a named region.	DPDP Act 2023 and Vaayu policy. BRD NFR-04, DR-007.
DP-5	Reuse what the client already operates. Introduce a new component only where it is genuinely differentiating.	Every new datastore is something Vaayu must run after we leave. Drives the pgvector and Prometheus choices in Section 4.
DP-6	Anything the client will want to tune is configuration, not code.	Change requests after go-live are constant. If tuning needs a developer, the system decays. Section 13.
DP-7	Every automated decision is reconstructable for 24 months.	Audit, dispute resolution and debugging turn out to be the same requirement. BRD FR-018, NFR-08.
DP-8	Degrade in tiers — full, reduced, manual — and never below the process that exists today.	The fallback for total failure is the current manual process, which still works. This is why the architecture is deliberately single-site. Section 7.

PRINCIPLES ARE HOW YOU WIN ARCHITECTURE ARGUMENTS

When a client architect asks why you are not using their preferred vector database, 'DP-5, and here is ADR-003' is a much stronger answer than a technical preference.

Write the principles first, get them agreed, and most of the individual decisions then argue for themselves.

PART A

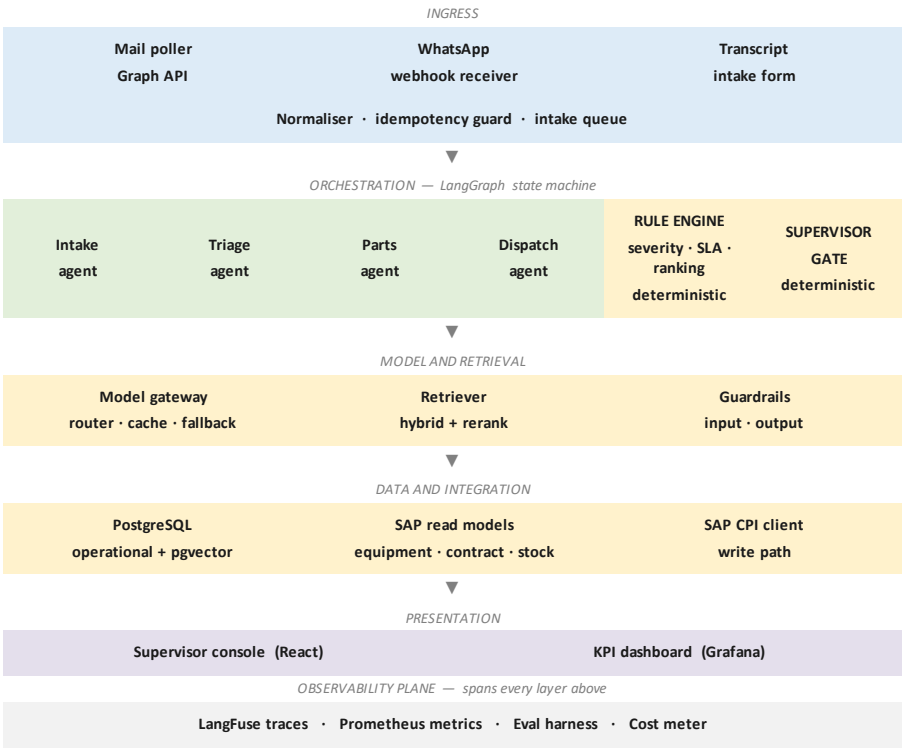
High-Level Design

Sections 3 to 8. Approved by client architecture and security. Answers: what is the system made of, what did we choose and why, where does it run, and how does it stay secure, available, fast and affordable.

3. Logical Architecture

3.1 Layered component view

Six layers. Data flows top to bottom on the request path; the observability plane spans all of them.



3.2 Component inventory

Component	Responsibility	Technology	Scaling
Channel adapters	Receive from three channels, normalise, enforce idempotency, enqueue	Python, FastAPI	3 replicas, stateless

Component	Responsibility	Technology	Scaling
Orchestrator	Run the LangGraph state machine; checkpoint every transition	LangGraph + PostgreSQL checkpointer	4 replicas, work queue
Agents (4)	Model-driven extraction, matching, diagnosis, parts prediction, ranking rationale	Python, Pydantic-validated outputs	In-process with orchestrator
Rule engine	Severity, SLA windows, technician score, parts shortfall — no model involved	Pure Python, table-driven, config-loaded	In-process
Supervisor gate	Apply routing rules; hold the review queue; sole holder of the SAP write tool	Pure Python	In-process
Model gateway	Route by task, serve from cache, enforce token and cost budgets, fall back on failure	LiteLLM-style proxy, self-hosted	2 replicas
Retriever	Hybrid search over manual corpus and service history, then rerank	pgvector + BM25, cross-encoder reranker	2 replicas
Guardrail filters	Input sanitisation and injection screening; output schema and leakage checks	Python, deterministic + classifier	In-process at gateway
Operational store	All entities in Section 9; also the vector index and the LangGraph checkpoints	PostgreSQL 16 with pgvector (existing cluster)	Existing HA pair
SAP integration clients	OData reads with caching; CPI channel for the BAPI write	Python httpx, tenacity retry	2 replicas
IoT consumer	Read the Event Hub endpoint, apply thresholds, raise alerts	Python, checkpointed consumer group	1 replica, checkpointed
Supervisor console	Review queue, recommendation detail, override capture	React + Vite, served by nginx	2 replicas
Observability plane	Traces, metrics, dashboards, cost meter, eval harness	LangFuse (self-hosted), Prometheus, Grafana	Existing cluster services

TWO BOXES ARE DELIBERATELY NOT AI

The rule engine and the supervisor gate are ordinary, boring, testable code. That is the point. Severity classification and technician ranking are decisions a business can write down, so they are written down and executed deterministically — which makes them auditable, cheap, instant and identical every time.

A common failure in enterprise AI projects is using a model for work that a table would do better. Ask of every model call: could this be a rule? If yes, make it a rule and spend the model budget where judgement is genuinely required.

4. Technology Stack and Rationale

4.1 Stack decisions

Every row names the alternatives that were genuinely considered and the trade-off accepted. Decisions with lasting consequence are written up in full as Architecture Decision Records in Annex B.

Layer	Chosen	Why	Trade-off accepted
Orchestration	LangGraph	Explicit state machine with durable checkpointing. Escalation timers must survive a pod restart, which a linear chain framework cannot guarantee.	Steeper learning curve for the Vaayu team at handover. Mitigated by the runbook and Section 13.
API framework	FastAPI	Async by default, and Pydantic validation is the same mechanism the TDD uses for agent output schemas. One validation model, not two.	Fewer batteries included than Django; the admin console is hand-built.
Relational store	PostgreSQL 16 (existing cluster)	Already operated by Vaayu's DBA team, already backed up, already in the DR plan. DP-5.	Shared cluster means noisy-neighbour risk; mitigated by a dedicated schema and connection limits.
Vector store	pgvector on the same cluster	340 manuals and 21,000 service orders is small. Adding a dedicated vector database would mean one more thing Vaayu operates after we leave. ADR-003.	Lower ceiling on scale and fewer index options. Revisit if the corpus grows beyond roughly ten times.
Model hosting	Azure OpenAI, Central India	Satisfies data residency without GPU procurement. ADR-001.	An external dependency and a network egress. Mitigated by the fallback chain in Section 7.2.
Model gateway	Self-hosted proxy	Routing, caching, budget enforcement and fallback belong in one place, not scattered through agent code.	One more service to run. Justified by it owning three cross-cutting concerns.
SAP write path	SAP Integration Suite (CPI)	Vaayu already licenses it, and IT policy forbids direct RFC from an application tier. BRD Section 4.2.	An extra network hop, roughly 200 ms. Irrelevant against a 90-second budget.
Observability	LangFuse, self-hosted	Traces contain raw complaint text, which is personal data and cannot leave the network. DP-4.	We operate it rather than consuming a managed service.
Metrics	Prometheus + Grafana (existing)	Already running, already on the Vaayu on-call rotation. DP-5.	None material.
Container platform	Existing on-premise Kubernetes (Rancher)	Already provisioned, already patched, already inside the network boundary.	Capacity is fixed at 12 vCPU / 48 GB per BRD ASM-03.
Secrets	Azure Key Vault	Already in the Vaayu tenant and already integrated with the cluster's CSI driver.	Cloud dependency for secret retrieval; cached with a TTL to survive a short outage.
CI/CD	GitHub Actions, self-hosted runner	Runner inside the Vaayu network so deployment never requires inbound access.	Runner maintenance is ours until handover.

Commented [SFR3]: The column that matters is the last one. Any architect can list technologies; what a client's architecture board actually wants to know is what you gave up.
A stack table with no trade-offs column reads as marketing. One with an honest trade-off in every row reads as engineering, and it is far harder to argue with.

4.2 Model routing policy

The model gateway classifies each task and routes it. This is architecture, not configuration, because it determines whether the system meets the cost ceiling in BRD NFR-09.

Commented [SFR4]: Routing is the single largest cost lever in an enterprise AI system, and it is an architecture decision, not a tuning exercise.
Note that the premium model appears exactly once. Most teams reach for the best model everywhere and discover the unit economics after go-live, when it is a renegotiation rather than a design choice.

Task	Routed to	Reason
Field extraction, fault classification	Small model	Closed-set work against a fixed schema. A frontier model adds cost and no accuracy.
Duplicate detection	Embedding model	Similarity comparison. No generation required at all.
Probable-fault diagnosis	Premium model	The only task requiring synthesis across retrieved manual sections and service history. The single justified use.
Rationale generation	Small model	Rewrites facts that have already been computed into plain language. No reasoning involved.
Severity, SLA, technician ranking	No model at all	Deterministic rules. Cheaper, faster, auditable, and identical every time.

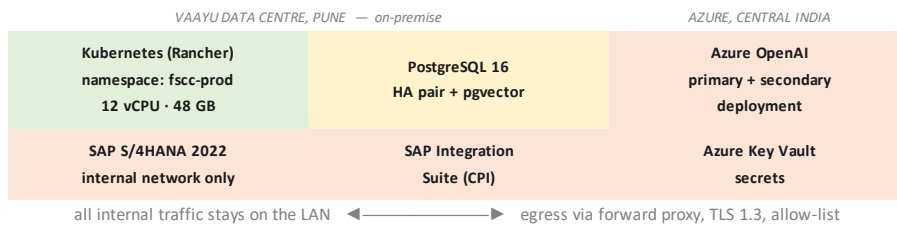
4.3 Rejected options

Option	Why it was rejected
On-premise LLM on Vaayu GPUs (vLLM)	GPU procurement lead time is 6 to 8 weeks against a 14-week engagement, and Vaayu has no GPU operations experience. Reconsider in Phase 2 when volume justifies it. ADR-001.
Dedicated vector database (Qdrant or Milvus)	A new datastore for a corpus this small violates DP-5. pgvector meets the requirement on infrastructure Vaayu already runs. ADR-003.
Direct RFC from the application to SAP	Forbidden by Vaayu IT policy, and it would place SAP credentials in the application tier.
Fine-tuning a model on Vaayu service history	The knowledge changes constantly and retrieval solves it without retraining. Fine-tuning would also fix an accuracy level in place that we could not then improve without a new training run.
Fully autonomous dispatch with no human gate	Contradicts DP-2 and BRD FR-009, and the client would not accept it for P1. Autonomy is earned after shadow mode, not assumed.
Multi-region active-active deployment	The fallback for total failure is the manual process that runs today. Paying for multi-region resilience to protect a system whose degraded mode is the status quo is poor value. Section 7.

5. Deployment Topology

5.1 On-premise and cloud split

Everything runs inside the Vaayu data centre in Pune except model inference and secret retrieval. There is exactly one egress path and it carries no personal data beyond the complaint text required for inference, which stays within the India region.



5.2 Network and egress

Path	Direction	Control
Application to SAP OData and CPI	Internal LAN only	Service account with least-privilege authorisation objects. No internet path exists.
Application to Azure OpenAI	Outbound only, via forward proxy	Destination allow-list restricted to the Central India endpoint. TLS 1.3. No inbound path.
Application to Microsoft Graph	Outbound only, via forward proxy	Azure AD application permissions, scoped to the single service mailbox.
WhatsApp Cloud API inbound webhook	Inbound to DMZ	Signature verification on every request; DMZ reverse proxy forwards to the cluster. The only inbound path in the system.
Azure IoT Hub	Outbound only	Managed identity, dedicated consumer group.
Console and dashboard users	Internal LAN and VPN	Azure AD SSO, role-based access per Section 6.2. Not exposed to the internet.

5.3 Environments

Environment	SAP connected to	Model endpoint	Purpose and promotion
DEV	SAP sandbox, and the public SAP Business Accelerator Hub sandbox before DEP-02 lands	Small model only, capped budget	Feature work. Promotes on green CI plus eval suite pass.
QA	SAP QAS	Both models, capped budget	Integration and regression. Promotes on full eval suite plus security scan.
UAT	SAP QAS with a production-like data copy	Production configuration	Business acceptance. Promotes on signed UAT exit.
PROD	SAP PRD	Production, full budget	Wave 0 shadow mode first, then Wave 1 write-back.

5.4 Sizing

Workload	Replicas	Request / limit	Basis
Channel adapters	3	0.3 / 0.6 vCPU, 512 MB	Poll and webhook, I/O bound
Orchestrator + agents	4	1.0 / 2.0 vCPU, 2 GB	40 complaints/hour peak, mostly waiting on model calls
Model gateway	2	0.5 / 1.0 vCPU, 1 GB	Proxy and cache lookups
Retriever	2	1.0 / 2.0 vCPU, 4 GB	Reranker is the memory driver
SAP clients + IoT consumer	3	0.3 / 0.6 vCPU, 512 MB	I/O bound
Console + dashboard	2	0.2 / 0.4 vCPU, 256 MB	Static serving
Total at limits	16 pods	≈ 11.4 vCPU, 34 GB	Within the 12 vCPU / 48 GB in BRD ASM-03, with headroom

SIZING IS A COMMITMENT, NOT AN ESTIMATE

BRD ASM-03 assumed 12 vCPU and 48 GB were available. This table proves the design fits inside that assumption, with the arithmetic shown.

If it did not fit, the correct move is not to shrink the design quietly — it is to raise it as a dependency with a six to eight week hardware lead time, in writing, while there is still time to act.

6. Security and Data Protection Architecture

6.1 Threat model summary

Threat	Vector	Architectural control
Prompt injection	Complaint text is written by anyone who can email or message the service number. It is untrusted input.	Structural, not instructional: complaint text is passed as delimited data; every agent has a tool allow-list; no model-driven component holds a write tool (DP-2).
Data exfiltration through the model	A crafted complaint attempts to make the model reveal another customer's data or the system prompt	Retrieval is scoped per request; output filter rejects responses echoing the system prompt; traces flag suspected extraction attempts.
Unauthorised SAP write	Compromise of the application tier	Service account holds only the authorisation objects for the four named interfaces. Order creation requires a supervisor identity on the request for P1.
Credential exposure	Secrets in code, environment or logs	Azure Key Vault via the cluster CSI driver. No secret in an environment variable. Log redaction on known key patterns.
Privilege escalation in the console	A supervisor acting on another depot's queue	Depot scoping enforced server-side on every query, not in the client. Section 6.2.
Supply chain	A compromised Python or npm dependency	Pinned lockfiles, dependency scanning in CI, self-hosted runner inside the network.

6.2 Identity and access

Azure AD SSO for all human users. Roles are enforced server-side on every request; the console never decides authorisation. Five roles are defined, with the full permission matrix in the TDD Section 9.3.

6.3 Data protection

Concern	Design	Traces to
Residency	All storage on-premise. Inference restricted to the Central India endpoint by proxy allow-list.	BRD NFR-04, DP-4
Personal data	Reporter name, phone and email encrypted at rest. Pseudonymised at 24 months while preserving audit linkage.	BRD DR-007
Data in traces	LangFuse self-hosted so raw complaint text never leaves the network. Audit records store input hashes rather than text.	BRD NFR-08, DP-4
Data in transit	TLS 1.2 minimum internally, TLS 1.3 on egress. mTLS on the CPI channel.	BRD NFR-06
Non-production data	UAT uses a production-like copy with personal fields pseudonymised before load.	DPO approval, 06 Feb 2026

7. Availability, Degradation and Recovery

7.1 Availability targets

Measure	Target	Design basis
Service availability	99.5% in business hours (07:00–22:00 IST, Mon–Sat)	BRD NFR-02. Outside business hours, best effort.
RPO (recovery point objective)	15 minutes	PostgreSQL WAL shipping on the existing cluster; no additional mechanism introduced.
RTO (recovery time objective)	4 hours	Redeploy from CI artefacts to the existing cluster. Manual triage covers the gap.
Architecture	Single site, no multi-region	Deliberate. See 7.3.

7.2 Degradation tiers

Each tier is announced to the coordinators; the system never degrades silently (DP-3).

Tier	Trigger	Behaviour	User sees
FULL	All dependencies healthy	Complete pipeline; auto-approval permitted above threshold	Normal operation
REDUCED — retrieval	Retriever or corpus unavailable	Triage continues without probable-cause diagnosis; all items route to review	Amber banner naming the missing capability
REDUCED — model	Primary model endpoint failing	Automatic failover to the secondary deployment; no change in behaviour	Nothing, unless both fail
REDUCED — SAP read	OData interfaces failing	Serve from the local read model with a staleness flag; route all to review	Amber banner with the data age
REDUCED — SAP write	CPI channel failing	Retry policy, then present the composed payload for manual IW31 entry	Red banner and the payload to copy
MANUAL	Both model endpoints failing, or the orchestrator down	Complaints queue; coordinators are alerted within 2 minutes and triage as they do today	Red banner and the queued complaint list

7.3 Why this architecture is deliberately single-site

The MANUAL tier is not a catastrophic failure. It is the process Vaayu runs today, at today's performance, with three coordinators who have done it for years and will still be employed. The worst case of a total outage is therefore a temporary return to a 47-minute triage time, not a stopped business.

A multi-region active-active deployment would roughly double infrastructure cost and add significant operational complexity to protect against an outcome the client already survives every day. That is poor value, and it is recorded here so the decision is not quietly reversed later by someone who has not thought it through.

This reasoning would not hold for a system with no manual fallback — a payments authorisation path, for instance. Availability architecture is a function of what happens when the system is absent, not of how important the system feels.

Commented [SFR5]: This is the most commercially useful paragraph in the document. The degraded mode of this system is the process the client runs today, which means the cost of an outage is bounded and known. That single observation is what justifies a single-site architecture and saves the client a large amount of money. Find the equivalent argument on your own engagements — it is usually there, and nobody has articulated it.

8. Latency, Cost and Observability Architecture

8.1 Latency budget

BRD NFR-01 allows 90 seconds at P95 from intake to recommendation. The budget is allocated per stage so that a regression can be attributed rather than argued about.

Stage	Budget (P95)	Measured, Wave 0	Notes
Ingress, normalise, enqueue	3 s	1.1 s	I/O bound
Intake agent (extraction)	12 s	8.4 s	Small model
Equipment match and SAP reads	10 s	4.2 s	Served from local read model
Retrieval and rerank	15 s	9.8 s	Hybrid search plus cross-encoder
Diagnosis (premium model)	30 s	19.6 s	The single largest stage
Parts check (SAP MM)	8 s	3.1 s	15-minute cache
Ranking, rationale, persist	12 s	6.9 s	Rules are instant; rationale is a small model
Total	90 s	53.1 s	Headroom retained for corpus growth

The supervisor console streams the recommendation as it is composed, so a supervisor begins reading before the pipeline completes. Time to first useful content matters more to a user than total completion time.

8.2 Cost architecture

BRD NFR-09 sets a ceiling of ₹28 per complaint. Four mechanisms hold the system under it, applied in this order.

#	Mechanism	How it works	Effect observed in Wave 0
1	Model routing	Only diagnosis reaches the premium model; four other tasks use a small model and three use no model at all. Section 4.2.	Largest single saving
2	Exact-response cache	Identical normalised complaint text within 24 hours returns the stored triage result.	Repeat rate ≈ 9%
3	Semantic cache	Near-identical complaints for the same equipment model share a diagnosis, subject to a similarity floor.	Additional ≈ 14% hit rate
4	Stable prompt prefix	System prompt and rule text are held constant at the front of every prompt; only the complaint varies at the end, so cached input pricing applies.	Reduces input token cost materially
5	Output caps and prompt hygiene	Per-task max output tokens (TDD Section 8); retrieved context capped at the top 5 reranked chunks.	Prevents slow cost drift

Commented [SFR6]: The caching ladder is ordered cheapest to most sophisticated for a reason: implement it in that order, and stop when the numbers are good enough. Enterprise service traffic is far more repetitive than teams expect. Measure your repeat rate before assuming you need the clever tier.

Cost is metered per complaint at the model gateway, exported to Prometheus, and alerted at ₹28 rolling daily mean and at 80% of the monthly budget. Cost is treated as a monitored service level, not as an invoice discovered at month end.

8.3 Observability and evaluation

Plane	What it captures	Used for
Tracing (LangFuse)	Every agent call: final prompt, retrieved chunk IDs, model, tokens, latency, confidence, tool calls, outcome	Reconstructing a decision (DP-7); diagnosing whether retrieval or generation failed
Metrics (Prometheus)	Throughput, stage latencies, auto-approval rate, override rate, SAP write failure rate, cost per complaint	Alerting and the KPI dashboard
Eval harness	200 labelled historical complaints; 40 adversarial injection cases; citation validity check	Runs in CI on every prompt, model or rule change. A failure blocks release.
Human feedback	Supervisor overrides with reason codes	New eval cases; the weekly review report; the correction loop in BRD FR-020

EVALS ARE THE UNIT TESTS OF AN AI SYSTEM

You cannot assert that output equals expected on non-deterministic text, so the equality assertion is replaced by a golden set scored against rubrics. Every prompt change, model swap or retrieval tweak runs against it before deploy.

The loop closes like this: a bad trace becomes a new eval case, the fix is verified against the suite, and only then does it ship. Teams that skip evals are not shipping faster — they are learning about their bugs from the client instead of from a dashboard.

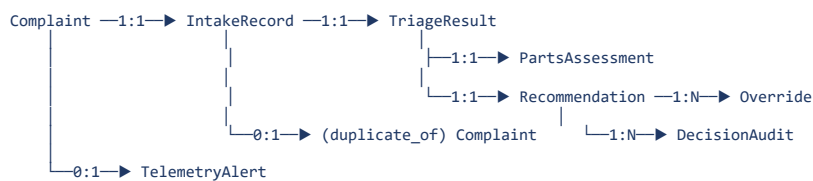
PART B

Low-Level Design

Sections 9 to 14. The structural input to the TDD: what the data looks like, what the interfaces are, how a request moves through the system, and what is configurable. Behaviour — rules, validation, error codes — is specified in the TDD.

9. Logical Data Model

9.1 Entity relationships



READ MODELS (projections of SAP, never authoritative – DP-1)

EquipmentReadModel	· refreshed nightly	· keyed by equipment_number
ContractReadModel	· cached 24 hours	· keyed by customer_id
MaterialStockCache	· cached 15 minutes	· keyed by material + storage_location
TechnicianRoster	· refreshed daily	· keyed by technician_id

9.2 Entities owned by this system

Entity	Purpose	Volume and retention
Complaint	The normalised inbound record, one per complaint on any channel	≈ 9,400/year. 24 months, then personal fields pseudonymised
IntakeRecord	Extracted structured fields, confidence and nulled-field flags. Immutable once written	1:1 with Complaint. 24 months
TriageResult	Equipment match, severity, the rule that fired, probable causes with citations, skill codes, SLA tier and due time	1:1 with IntakeRecord. 24 months
PartsAssessment	Predicted parts with likelihood bands, stock position and the time it was read	1:1 with TriageResult. 24 months
Recommendation	Ranked technicians, selection, confidence, rationale, routing reason and status	1:1 with TriageResult. 24 months
Override	Supervisor amendments and rejections with reason codes. The original is never mutated	≈ 20% of recommendations. 24 months
DecisionAudit	One row per agent invocation: model, prompt version, input hash, retrieved source IDs, output, confidence, latency, cost	≈ 5 per complaint. 24 months
TelemetryAlert	Threshold breaches from IoT, optionally linked to a raised complaint	Variable. 90 days

9.3 Read models

The four read models are projections of SAP data held locally for latency and for graceful degradation. They are never authoritative (DP-1): a disagreement between a read model and SAP is resolved in SAP's favour, and every read model row carries the timestamp it was refreshed so staleness is visible rather than assumed.

10. Interface Inventory and Contracts

10.1 Internal API surface

All endpoints are versioned under /v1, authenticated with Azure AD bearer tokens, and return JSON. Request and response models are named here; field-level rules, error codes and retry policy are specified in TDD Section 4.

Endpoint	Method	Request / response model	Traces to
/v1/complaints	POST	ComplaintCreate → ComplaintAccepted	BRD FR-001
/v1/complaints/{id}	GET	— → ComplaintDetail	BRD FR-001, FR-002
/v1/complaints/{id}/triage	GET	— → TriageResultView	BRD FR-005, FR-006
/v1/queue	GET	QueueFilter → QueuePage	BRD FR-019
/v1/recommendations/{id}	GET	— → RecommendationDetail	BRD FR-011
/v1/recommendations/{id}/approve	POST	ApprovalRequest → WorkOrderCreated	BRD FR-017
/v1/recommendations/{id}/override	POST	OverrideRequest → OverrideAccepted	BRD FR-012, FR-020
/v1/audit/{work_order_id}	GET	— → DecisionTrail	BRD FR-018
/v1/config/thresholds	GET, PUT	ThresholdConfig ↔ ThresholdConfig	BRD FR-009
/healthz, /readyz	GET	— → HealthStatus	BRD NFR-02

10.2 External interface inventory

ID	System	Direction	Protocol	Client-side owner
INT-001	SAP PM equipment master and functional location	Read	OData v2, OAuth 2.0	R. Balan
INT-002	SAP PM maintenance order create	Write	BAPI via CPI, OAuth 2.0 + mTLS	R. Balan
INT-003	SAP MM material stock at depot	Read	OData v2, OAuth 2.0	R. Balan
INT-004	SAP SD service contract and SLA tier	Read	OData v2, OAuth 2.0	R. Balan
INT-005	Microsoft 365 service mailbox	Read	Graph API, app permissions	F. Qureshi
INT-006	WhatsApp Business Cloud API	Read / Write	Webhook in, template out	A. Deshpande
INT-007	Azure IoT Hub telemetry	Read	Event Hub endpoint, managed identity	F. Qureshi
INT-008	Equipment manual corpus	Read	SFTP delta drop	R. Balan

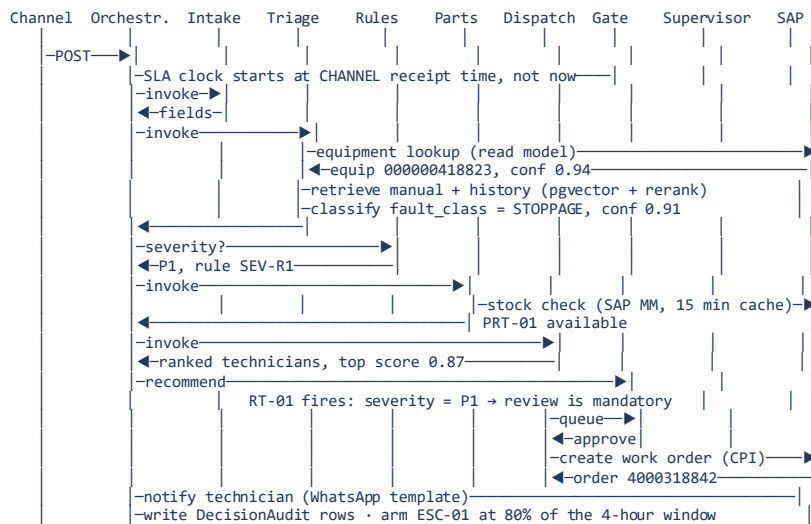
WHY THE OWNER COLUMN IS NOT DECORATION

Six of these eight interfaces cannot be built by you. They require someone inside the client to enable a service, grant an authorisation object or provision a subscription.

INT-002 took eleven working days to enable on this engagement, and that was survivable only because it was asked for in week one with a named owner and a date. An interface table without owners is a list of things that will surprise you later.

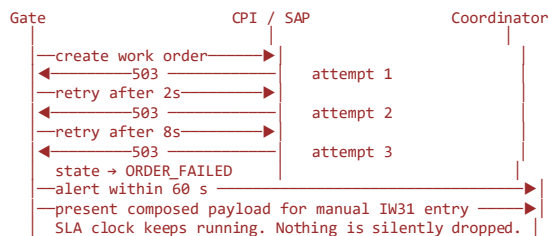
11. Sequence Designs

11.1 P1 complaint to work order — the approved path



Elapsed time from POST to the item appearing in the review queue: 41 seconds measured in Wave 0.
Supervisor response time sits outside the NFR-01 budget and is measured separately.

11.2 SAP write failure — the degraded path



DRAW THE FAILURE SEQUENCE, NOT JUST THE HAPPY ONE

Most design documents contain one sequence diagram and it is the happy path. The client's security and operations reviewers will ask about the other one, and the answer is far more revealing.

Note what does not happen here: the work order is not silently lost, the SLA clock is not reset to hide the delay, and the failure is not swallowed into a log nobody reads. DP-3 in practice.

12. Agent and Tool Boundary Design

The pipeline is a fixed workflow with four model-driven steps, not an autonomous agent choosing its own route. The sequence of stages is determined by the state machine in TDD Section 5.3; the model supplies judgement within a stage, never control over the flow.

Agent	Responsibility	Tools held	Explicitly cannot
Intake	Normalise, extract structured fields, detect duplicates	extract_fields, find_open_orders	Assign severity. Reach any SAP write path.
Triage	Identify equipment, diagnose probable fault with citations, determine skill codes	sap_equipment_lookup, retrieve_manual, retrieve_history, sap_contract_lookup	Select a technician. Create anything in SAP.
Parts	Predict probable parts, check depot stock, propose alternatives	sap_stock_lookup, depot_distance	Create a stock transfer or reservation.
Dispatch	Produce a ranked recommendation and a plain-language rationale	technician_roster, distance_matrix	Approve its own recommendation. Write to SAP.
Rule engine	Severity, SLA windows, technician score, parts shortfall	None — pure functions over configuration	Be overridden by a model.
Supervisor gate	Apply routing rules, hold the queue, execute the approved write	sap_order_create	Bypass the P1 rule. It is deterministic code and exercises no judgement.

Commented [SFR7]: Every extra autonomous step compounds error. At 95% reliability per step, ten chained steps land near 60% end to end. That arithmetic is why this design is a fixed workflow with model-driven steps inside it, rather than an open-ended agent deciding its own path. Use open-ended agency where the task is genuinely open-ended; use a workflow where the process is already known — and in an enterprise, the process is almost always already known.

THE ONE-SENTENCE SECURITY ARGUMENT

sap_order_create is held by exactly one component, and that component contains no model. Therefore no text arriving from outside the company — however crafted — can cause a work order to be created.

If you can say a sentence of that shape about every irreversible action in your architecture, the security review is largely over. If you cannot, no amount of prompt engineering will substitute for it.

13. Configuration Inventory

DP-6 requires that anything the client will want to tune is configuration rather than code. This is the complete list, with who may change it and whether a redeploy is required.

Setting	Default	Changeable by	Redeploy needed?
Recommendation confidence threshold	0.85	Administrator, in the console	No
Equipment match threshold	0.80	Administrator, in the console	No
Technician ranking weights	0.45 / 0.30 / 0.25	Administrator, in the console	No
Maximum dispatch radius	150 km	Administrator, in the console	No
SLA windows by tier	Section 6.2 of the TDD	Administrator, with dual approval	No
Escalation recipients per depot	Depot supervisor + Head of CS	Administrator, in the console	No
Severity rule table	SEV-R1 to SEV-R9	AtliQ, via a versioned config file	No — hot reloaded, change logged
Model routing policy	Section 4.2	AtliQ only	Yes
Prompt templates	Versioned in the repository	AtliQ only	Yes — and must pass the eval suite
Cost alert thresholds	₹28 per complaint, 80% of monthly budget	Administrator	No

WHERE THE LINE SITS, AND WHY

Thresholds, weights, windows and recipients are configuration, because the client will want to tune them in the first month and should not need us to do it.

Prompts and routing are not, because changing them changes system behaviour in ways that must pass the eval suite before reaching production. Making something configurable is a decision about who is accountable when it is wrong.

14. Non-Functional Traceability

Every non-functional requirement in the BRD maps to a design decision here. This table is how the client's architecture board verifies that the design actually satisfies what was signed.

BRD ref.	Requirement	Satisfied by
NFR-01	Triage within 90 s at P95	Section 8.1 latency budget with per-stage allocation; async pipeline; streaming console
NFR-02	99.5% availability in business hours	Section 7.1; replica counts in Section 5.4; degradation tiers in 7.2
NFR-03	40 complaints/hour, graceful under load	Queue-based ingress in Section 3.1; orchestrator replicas in 5.4; degrades by queueing, never by dropping
NFR-04	Data residency in India	Section 5.1 on-premise split; proxy allow-list in 5.2; self-hosted tracing in 6.3
NFR-05	On-premise deployment	Section 5.1; only inference and secret retrieval egress
NFR-06	Security — secrets, RBAC, TLS	Section 6.1 threat model; 6.2 identity; 6.3 data protection
NFR-07	Prompt injection resistance	Section 12 tool boundaries; DP-2 deterministic gate; guardrail filters in 3.2; adversarial eval suite in 8.3
NFR-08	24-month auditability	DecisionAudit entity in Section 9.2; tracing plane in 8.3
NFR-09	≤ ₹28 per complaint	Section 8.2 five cost mechanisms; routing policy in 4.2; metered and alerted at the gateway
NFR-10	Model fallback	Section 7.2 degradation tiers; secondary deployment in 5.1
NFR-11	Usable after 45 minutes of training	Console design in Section 3.2; queue sorted by SLA remaining; rationale rendered in plain language

Annex A — Diagram Source

The diagrams in this document were drawn as Word tables so the file stays self-contained. On a live engagement you would produce them in draw.io, Eraser.io or Mermaid and paste the image. The source below regenerates each one — use it, do not redraw by hand.

A.1 System context — Mermaid

```
graph TD
    subgraph TB
        subgraph CH [Inbound channels]
            M[Service mailbox<br/>Microsoft 365]
            W[WhatsApp<br/>Business Cloud API]
            P[Phone transcript]
        end
        subgraph SOR [Systems of record]
            SAP["SAP S/4HANA 2022<br/>PM · MM · SD"]
            IOT["Azure IoT Hub<br/>2300 pumps"]
        end
        FSCC["AI Field Service Command Centre"]
        CO[Service coordinators]
        SU[Depot supervisors]
        TE[Field technicians]
        M --> FSCC
        W --> FSCC
        P --> FSCC
        SAP <--> FSCC
        IOT --> FSCC
        FSCC --> CO
        FSCC --> SU
        FSCC --> TE
    end
```

A.2 Component architecture — Eraser.io DSL

```
direction right

Ingress [color: blue] {
  Mail poller; WhatsApp receiver; Transcript form; Normaliser + idempotency guard
}
Orchestration [color: green] {
  Intake agent; Triage agent; Parts agent; Dispatch agent
  Rule engine [color: orange]; Supervisor gate [color: orange]
}
Model and retrieval [color: yellow] {
  Model gateway (router + cache + fallback); Retriever (hybrid + rerank); Guardrails
}
Data [color: yellow] {
  PostgreSQL + pgvector; SAP read models; SAP CPI client
}
Presentation [color: purple] { Supervisor console; KPI dashboard }
Observability [color: gray] { LangFuse; Prometheus; Eval harness; Cost meter }

Ingress > Orchestration > Model and retrieval > Data
Orchestration > Presentation
Observability > Orchestration
```

A.3 P1 sequence — Mermaid

```
sequenceDiagram
    participant CH as Channel
    participant OR as Orchestrator
    participant TR as Triage agent
    participant RU as Rule engine
    participant GA as Supervisor gate
    participant SU as Supervisor
    participant SAP as SAP PM
    CH->>OR: POST /v1/complaints
    Note over OR: SLA clock starts at channel receipt time
```

```
OR->>TR: triage(intake)
TR->>SAP: equipment lookup (read model)
SAP-->>TR: equipment 00000418823, conf 0.94
TR-->>OR: fault_class STOPPAGE, conf 0.91
OR->>RU: severity?
RU-->>OR: P1 (SEV-R1)
OR->>GA: recommendation
Note over GA: RT-01 fires - P1 review is mandatory
GA->>SU: queue item
SU-->>GA: approve
GA->>SAP: create work order (CPI)
SAP-->>GA: order 4000318842
```

Annex B — Architecture Decision Records

An ADR records one decision, the alternatives genuinely considered, and what following it costs. Three are recorded here. They exist so that in month four, when someone asks why the system is built this way, the answer is a document rather than an argument.

ADR-001 — Model inference in Azure OpenAI Central India rather than on-premise

Status	Accepted, 16 Feb 2026. Supersedes the on-premise proposal in SDD v0.2.
Context	BRD NFR-04 requires that personal data and inference stay within India. Vaayu's security team initially preferred fully on-premise hosting on their own GPUs. The engagement is 14 weeks and Vaayu holds no inference-capable GPUs today.
Decision	Use Azure OpenAI in the Central India region, reached through the existing forward proxy with a destination allow-list. No inference leaves the region. On-premise hosting is deferred to Phase 2.
Consequences	Residency is satisfied without hardware procurement. Introduces one external dependency and one egress path, both mitigated by the degradation tiers in Section 7.2. Per-token cost applies, which is why the routing and caching design in Section 8.2 is part of the architecture rather than an optimisation.
Rejected	On-premise vLLM on new GPUs — 6 to 8 week procurement lead time against a 14-week engagement, plus GPU operations skills Vaayu does not yet have. Reconsider in Phase 2 when steady-state volume is known.

ADR-002 — A deterministic gate holds the only SAP write capability

Status	Accepted, 11 Feb 2026.
Context	Complaint text is untrusted input written by anyone who can email or message the service number. BRD NFR-07 requires that the system not execute instructions embedded in a complaint. Instruction-based defences alone are not testable to a zero-failure standard.
Decision	The sap_order_create tool is held exclusively by the supervisor gate, which contains no language model. No model-driven agent holds any write capability. DP-2.
Consequences	Prompt injection becomes structurally incapable of creating a work order rather than merely discouraged, which makes NFR-07 verifiable. Costs one extra hop in the pipeline and removes the option of fully autonomous dispatch, which BRD Section 4.2 excludes anyway.
Rejected	Guardrail prompts alone — not verifiable to a zero-failure bar. A permissions layer inside the agent — still places the capability behind a non-deterministic decision.

ADR-003 — pgvector on the existing PostgreSQL cluster rather than a dedicated vector database

Status	Accepted, 12 Feb 2026.
Context	The corpus is 340 manuals plus 36 months of service orders, roughly 21,000 documents after chunking. Qdrant and Milvus were both proposed. Vaayu must operate whatever we leave behind, and their platform team supports PostgreSQL today and nothing else in this class.
Decision	Use pgvector on the existing HA PostgreSQL pair, in a dedicated schema with its own connection limits. DP-5.
Consequences	No new datastore to run, back up, patch or hand over. Retrieval quality is met at this corpus size with hybrid search plus reranking. Ceiling is lower than a purpose-built vector database; the trigger for revisiting is a corpus roughly ten times larger or P95 retrieval latency above 15 seconds.
Rejected	Qdrant and Milvus — better scaling characteristics, but both add an operational burden that outlasts the engagement for a corpus that does not need them.

Annex C — SDD Reviewer's Rubric

Grade your own solution design before it reaches a client architecture board. Any No in the Blocking column means it is not ready to present.

#	Check	Blocking	Where here
1	Are there written design principles, agreed before the decisions, that later sections can appeal to?	Yes	Section 2.3
2	Does the design say what it is defending against — specifically, which AI property forces each unusual component?	Yes	Section 2.2
3	Does every technology choice name the alternatives considered and the trade-off accepted?	Yes	Section 4.1
4	Is there a model routing policy, rather than one model used for everything?	Yes	Section 4.2
5	Is any work being done by a model that a deterministic rule would do better?	Must be No	Sections 3.2, 4.2
6	Does the deployment topology state exactly what egresses and to where?	Yes	Sections 5.1, 5.2
7	Does the sizing arithmetic actually fit the capacity assumed in the BRD, shown rather than asserted?	Yes	Section 5.4
8	Is there a threat model naming prompt injection as an untrusted-input problem?	Yes	Section 6.1
9	Are degradation tiers defined, and does every tier announce itself to the user?	Yes	Section 7.2
10	Is the availability target justified by what happens when the system is absent, not by how important it feels?	Yes	Section 7.3
11	Is there a latency budget allocated per stage, so a regression can be attributed?	Yes	Section 8.1
12	Is cost a designed property with named mechanisms, and is it metered and alerted?	Yes	Section 8.2
13	Is there an eval harness in CI that blocks release, and is it named as such?	Yes	Section 8.3
14	Can any model-driven component reach an irreversible action?	Must be No	Section 12, ADR-002
15	Does every external interface name a client-side owner?	Yes	Section 10.2
16	Does every non-functional requirement in the BRD map to a specific design decision?	Yes	Section 14
17	Are the consequential decisions written as ADRs with rejected alternatives?	No	Annex B
18	Is diagram source included, so the next person can regenerate rather than redraw?	No	Annex A

ROWS 5 AND 14 ARE THE TWO THAT MATTER MOST

Row 5 is about judgement: enterprise AI projects routinely spend a premium model on work a lookup table would do faster, cheaper and more auditably. Every model call should have to justify itself.

Row 14 is about safety, and it is the only question a security reviewer will not negotiate on. Put a deterministic gate in front of every irreversible action, and you can answer it in one sentence.